11. LLD of Car Rental System

https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/interviewquestions/carrental?ref_type=heads

Lets understand the flow first:

1. User comes to care rental system.
2. Select a particular Store based on its Location.
3. Search for Vehicles based on vehicle type(like "Four Vehicle", "Two Wheeler" etc.) and its availability.
4. Reserve the selected vehicle.
   a. Should handle concurrency, as 2 users try to reserve the same vehicle for same time.
5. Start the trip.
6. Submit the vehicle.
7. Bill is generated against the reservation.
   a. We can have multiple bill strategies for generating the bill, like daily Bill strategy or hourly bill strategy etc.
8. Payment is made against the Bill.
   a. We can have multiple payment strategies for processing the payment like UPI, Card etc.

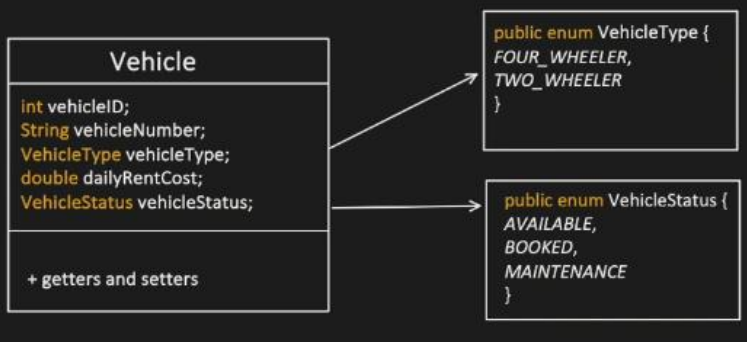
Based on above understanding, we identified the below major objects involved in this design:

- Vehicle
- Store
- Reservation
- User
- Bill
- Payment

Object has properties and behavior.

Lets start creating the UML:

1. Vehicle



For Each Store there is separate Vehicle – Inventory – Manager (not a single ton class)

Vehicle object is not intelligent, its just a POJO. We need some other intelligent object, which can manage Vehicle inventory, like:

- Maintain overall vehicles list.
- Maintain booked vehicles list.
- Add vehicle
- Remove vehicle
- Check for vehicle availability

etc.

That's where VehicleInventoryManager comes into the picture.

Note:
each store has their own VehicleInventoryManager, so it is not a singleton class

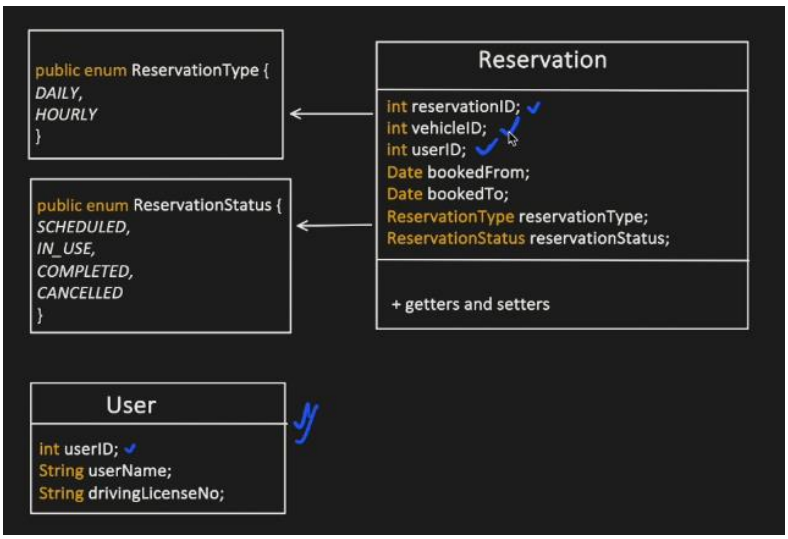
2. VehicleInventoryManager

We will enhance the capability of this VehicleInventoryManager later when Reservation capability will get added.

Vehicle Inventory Manager has all the vehicles in that store

3. Reservation

Reservation, does not need full Vehicle and User details, so we can keep their ids itself in it.



Now, Reservation is not smart enough to handle capabilities like:

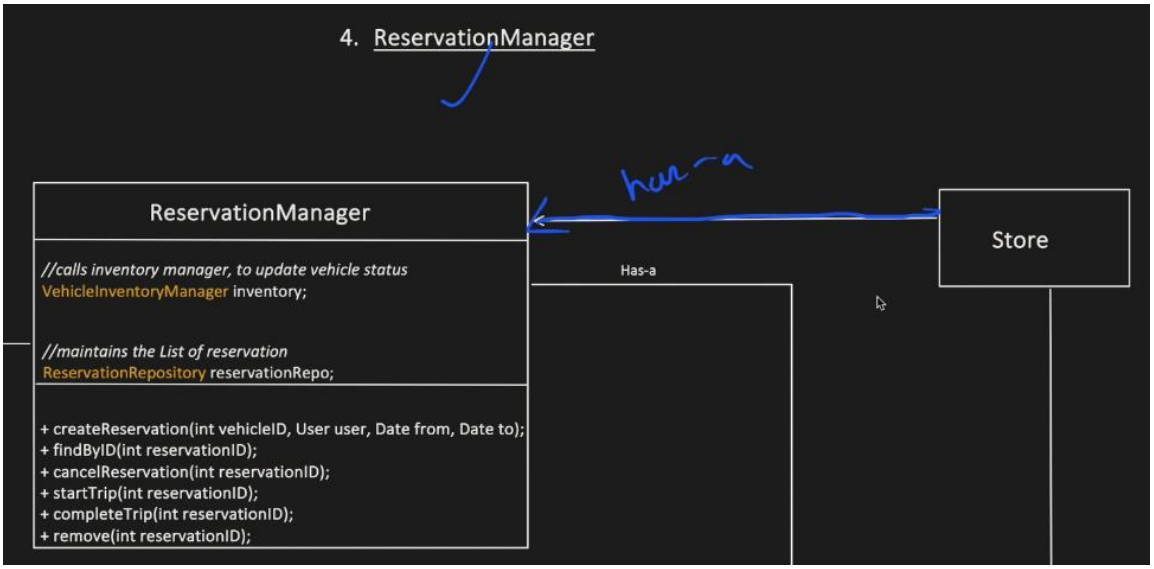
- Maintaining the List of Reservations.
- Fetch particular reservation details.
- Creation of reservation, changing the reservation status.
- Cancellation of reservation, changing the reservation status.
- Remove the reservation

  Etc.

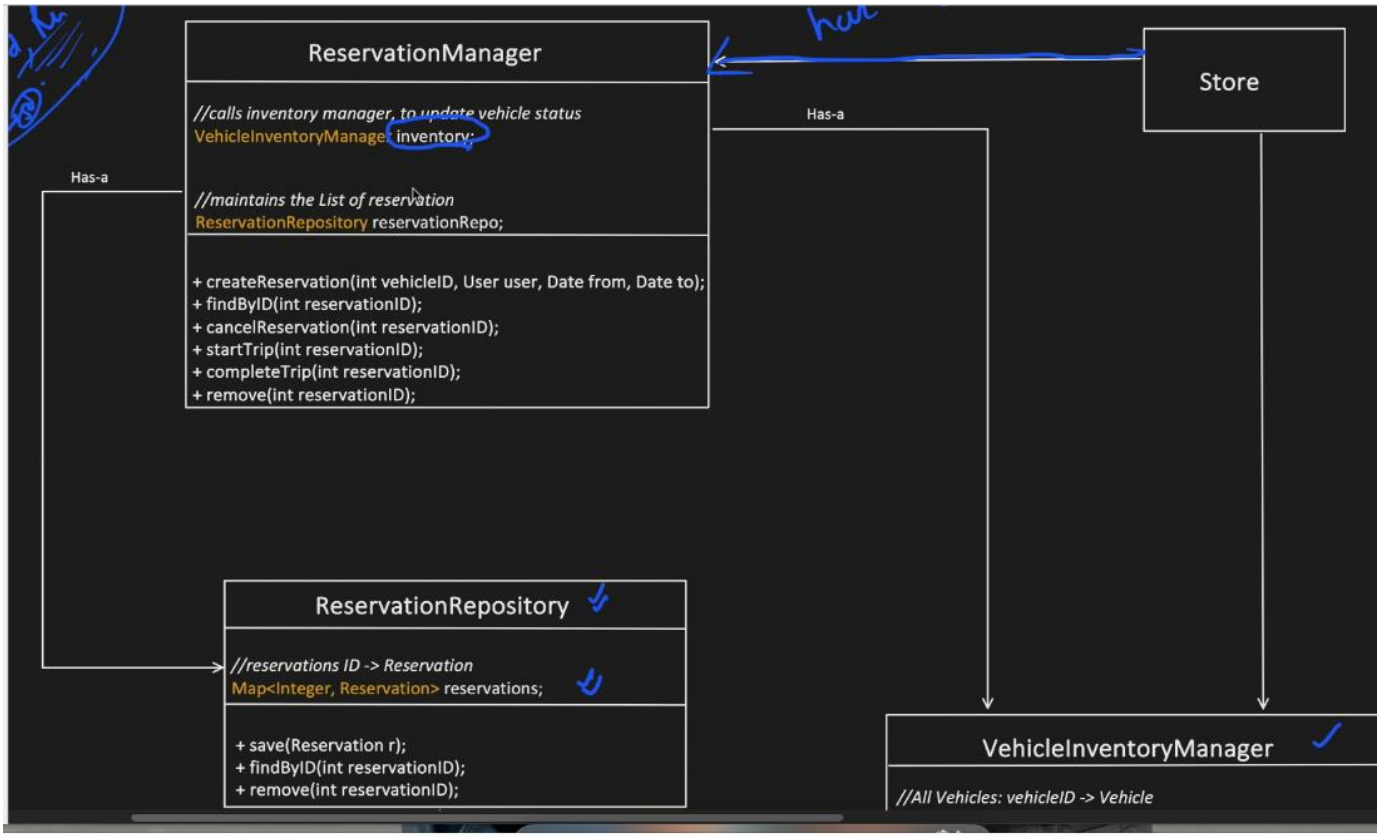
That's where ReservationManager comes into the picture.

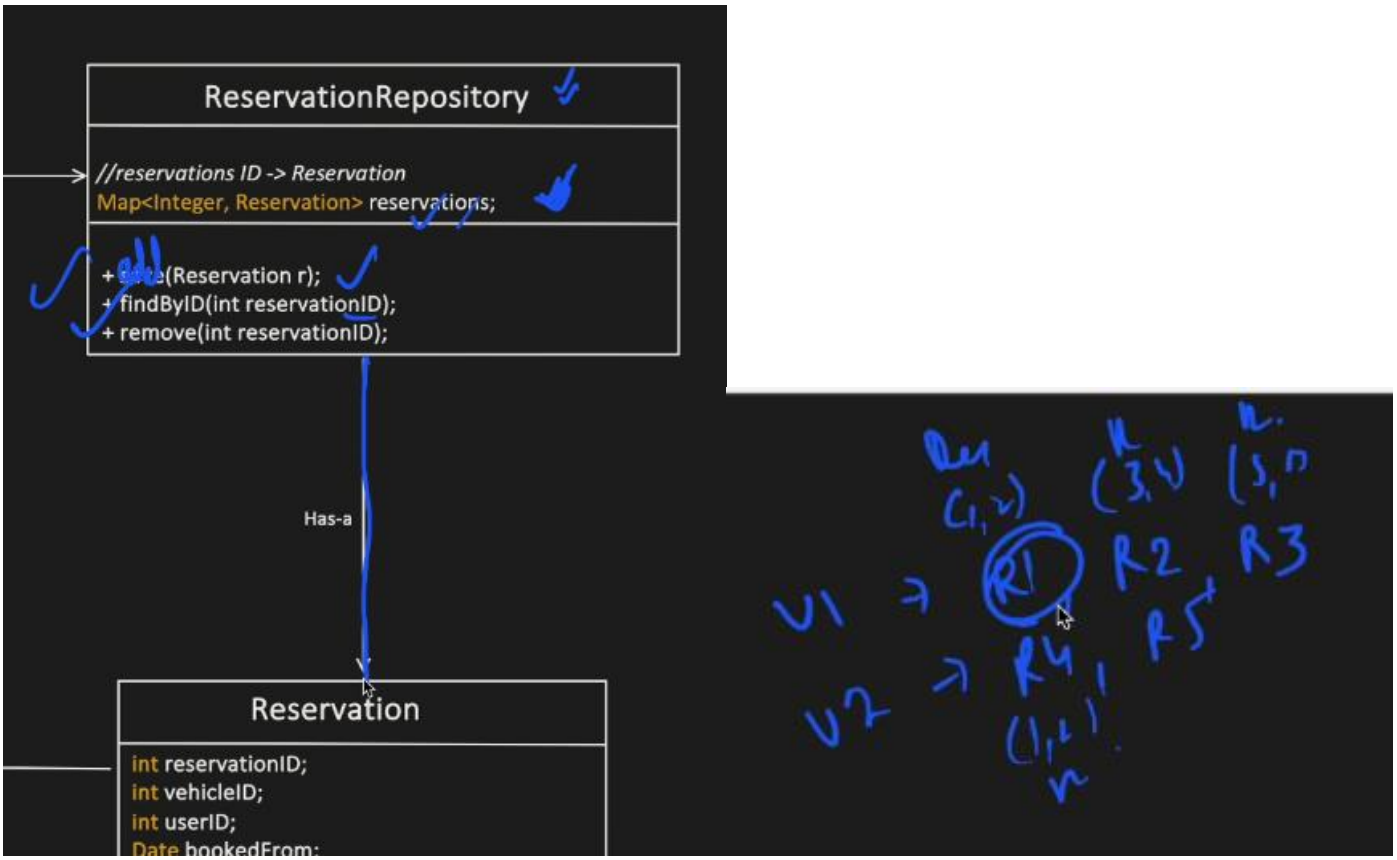
Note:
each store has their own ReservationManager, so it is not a singleton class

4. ReservationManager



Reservation manager tells the Vehicle manager to update the status of Vehicle.

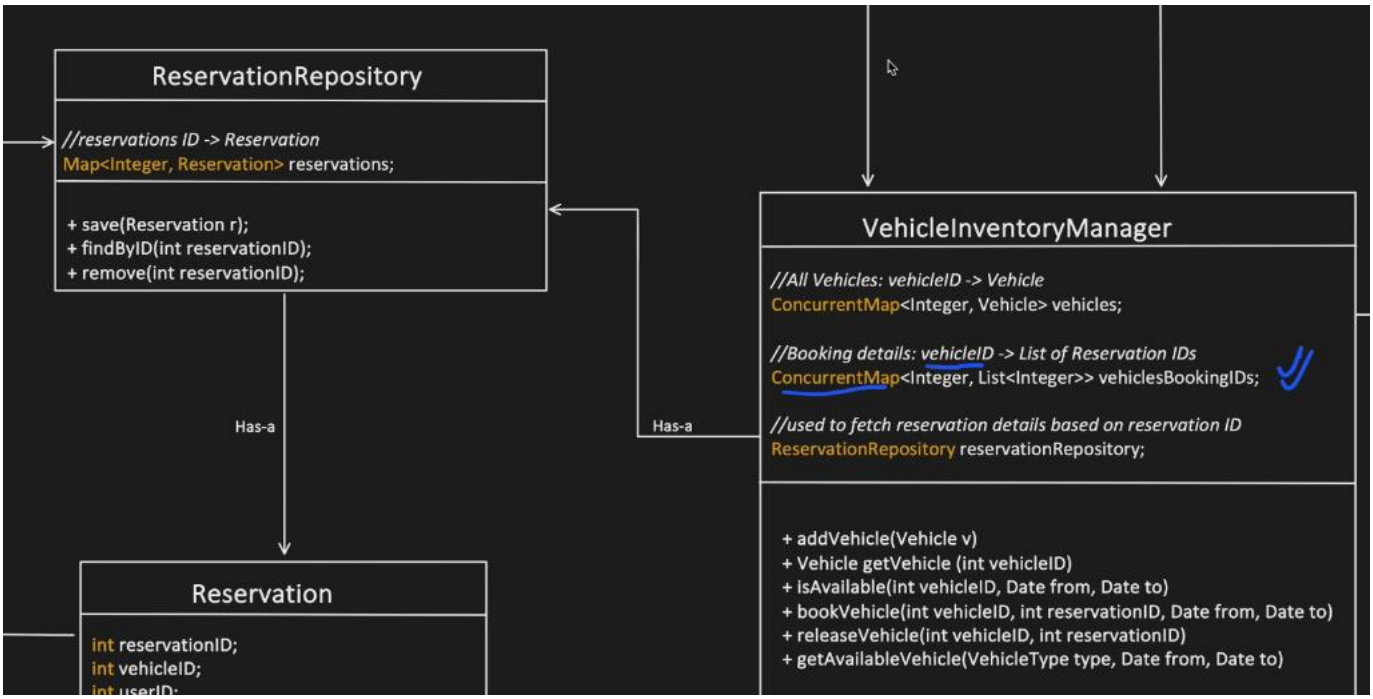




Reservation repository is used to remove circular dependency.

Need to handle:
Create Reservation should only be successful, when vehicle is available on that particular date.

Whats the right place to hold the vehicle booking information. Its "VehicleInventoryManager"



Note:
I created a ReservationRepository to break the circular dependency between ReservationManager and VehicleInventoryManag

We have a map of vehicle ID and the list of reservations ID (in VehicleInventoryManager), so, one vehicle can have multiple reservations
So, this ReservationRepository has this information (booked from- to and everything).
So if I only keep reservation ID itself in VehicleInventoryManager , from there I can take out the information for the reservation. I can take out what the date it is booked from to till.
So that's where I have put vehicle ID versus the list of reservation IDs, because this will help me to find out the available vehicles on a particular day, which user has selected.
When you have a reservation ID , using this reservation ID, you have to fetch the particular reservation and if ReservationManager holds the all reservation info then VehicleInventoryManager need to have a dependency of this ReservationManager. And then there would be a circular dependency.
ReservationManager ---has a---> VehicleInventoryManager ---has a---> ReservationManager
So what I have done is I have introduced a ReservationRepository which has all the reservations.

Another thing which need to be handle is: Concurrency

2 important things to consider while handling the concurrency:

1st: Collections like List of vehicles, List of Booked vehicles should be able to handle concurrency. So that our underlying data structu consistent when multiple threads try to read or write in it.

so that our underlying data structures should be

2nd: Need capability, to put lock per Vehicle, this is needed for achieve atomicity.

1st concurrent threads are multiple threads of execution running within a single program, either simultaneously (on multi-core CPUs) or interleaved in time (on a single CPU). We always do in memory, so it's better to have your collection concurrent so that underlying data structure should be consistent when multiple threads try to read and write in it.
2nd atomicity means that a set of operations are treated as a single, indivisible unit of work; they either all complete successfully, or none of them do, even in the event of concurrent access or system failure.

Assume both thread one and thread two are running concurrently.
And, in parallel, threads come and check for vehicle availability.
So the system returns - Yes, vehicle one is available - to both
And both selected are same vehicle.
Now they both try to reserve this particular vehicle V1.So when they both try to reserve it.
Our collection is concurrent, which means only one operation at a time can do.
So here, if you see first thread one, try to reserve V1. it would be able to do the reservation - R1
While thread two might be waiting because vehicle a concurrent map.
And once R1 is done and when thread two get a chance, it will also try to reserve the same vehicle.
And it will create its own reservation - R2.
So even so for the same vehicle two reservation is created for the same day - problem.
So you understand that just by using the concurrent map is not sufficient in this scenario.
So we have to handle this concurrency in this scenario by putting a lock.

2nd: Need capability, to put lock per Vehicle, this is needed for achieve atomicity.

For example: assume both thread1 and thread2 running concurrently, below scenario will cause an issue:

Thread1:
- Check for vehicle availability, touches the List of vehicles
- Select a particular vehicle : V1
- Try to reserve this particular vehicle (V1), change the status of vehicle
- Update the List of Booked vehicles

Thread2:
- Check for vehicle availability, touches the List of vehicles
- Select a particular vehicle: V1
- Try to reserve this particular vehicle (V1), change the status of vehicle
- Update the List of Booked vehicles

Now, even though our collections are concurrent (only 1 thread can touch it at a time), still above scenario will cause an issue and bot thread will be able to book the same vehicle for same slot.

To handle it, we need to bring lock per vehicle:

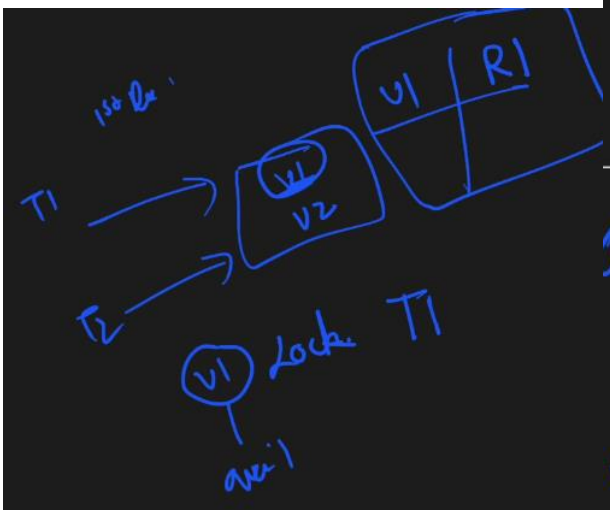
To handle it, we need to bring lock per vehicle:

Thread1:
- Check for vehicle availability, touches the List of vehicles
- Select a particular vehicle : V1
- Put a lock on V1
- Check for availability again
- Try to reserve this particular vehicle(V1), change the status of vehicle
- Update the List of Booked vehicles
- Release lock

Thread2:
- Check for vehicle availability, touches the List of vehicles
- Select a particular vehicle: V1
- Put a lock on V1
- Check for availability again
- Try to reserve this particular vehicle, change the status of vehicle
- Update the List of Booked vehicles
- Release lock

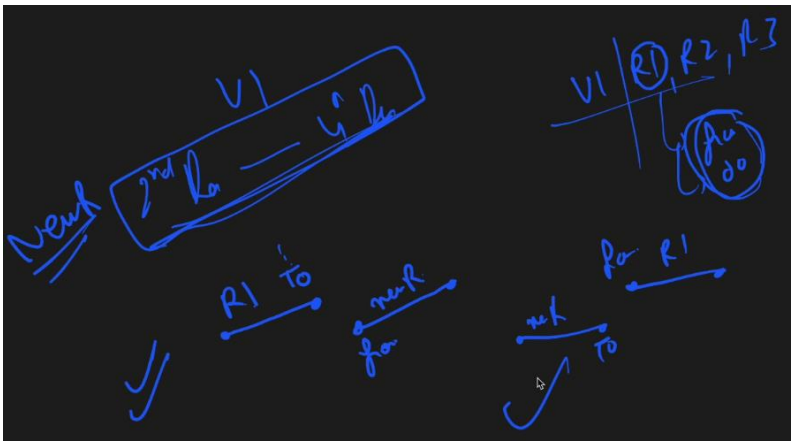
Now, one thread say Thread2 will wait for the lock to release on V1.
When Thread1, release the lock, then Thread2 will proceed and check for availability again, but this time it will not be able to reserv available.

Udemy



```
VehicleInventoryManager

//All Vehicles: vehicleID -> Vehicle
ConcurrentMap<Integer, Vehicle> vehicles;

//Booking details: vehicleID -> List of Reservation IDs
ConcurrentMap<Integer, List<Integer>> vehiclesBookingIDs;

//used to fetch reservation details based on reservation ID
ReservationRepository reservationRepository;

//Lock per vehicle: vehicleID -> Lock
ConcurrentMap<Integer, ReentrantLock> vehicleLocks;

+ addVehicle(Vehicle v)
+ Vehicle getVehicle (int vehicleID)
+ isAvailable(int vehicleID, Date from, Date to)
+ bookVehicle(int vehicleID, int reservationID, Date from, Date to)
+ releaseVehicle(int vehicleID, int reservationID)
+ getAvailableVehicle(VehicleType type, Date from, Date to)
+ putLockOnVehicle(int vehicleID)
```

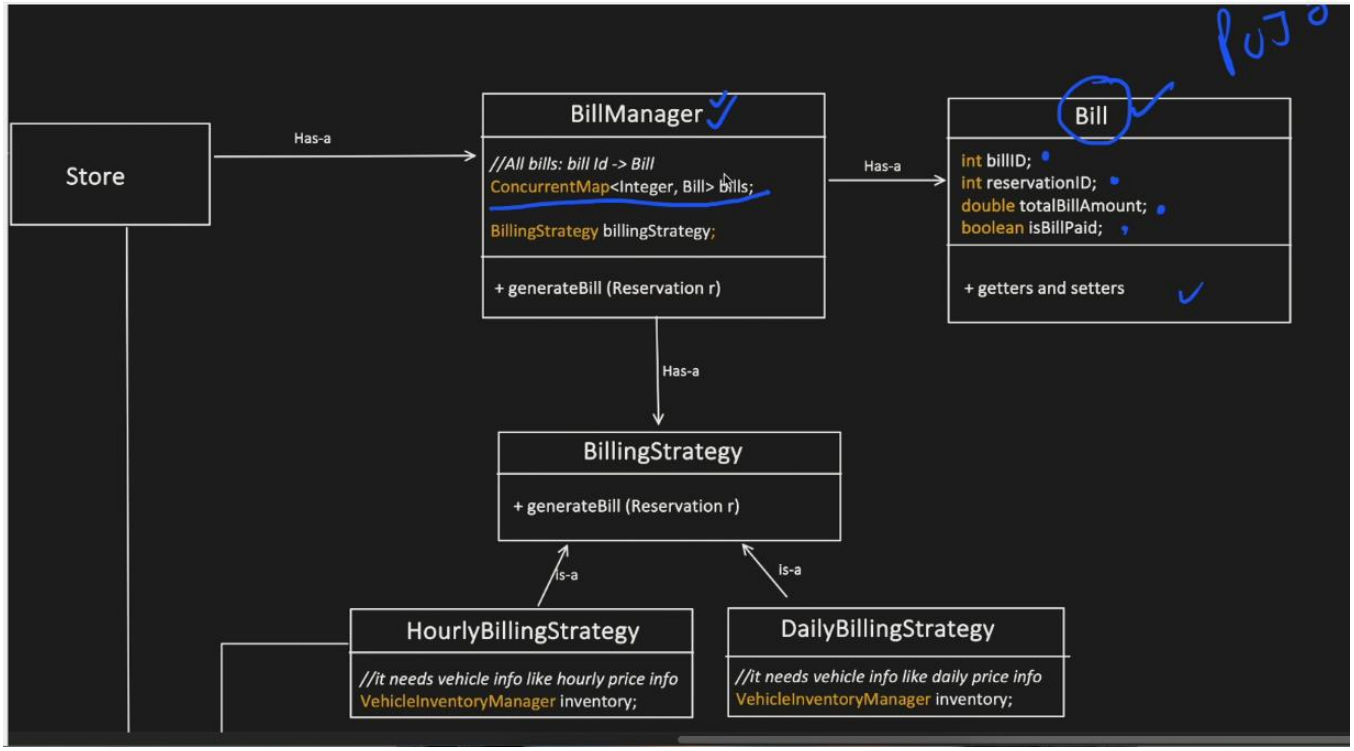
Now let's say again the same system; two threads come into this place. Look for availability for 1st December. The system says V1. both select a particular vehicle.
Now anyone who picks a particular vehicle, puts a lock.
thread 1 has acquired the lock thread, now it will check for availability again?
Yes, currently it is available. In the map against the vehicle - there is no reservation for that day. So it's available. it creates a new reservation and updates the reservation book list. And then it releases the lock.
Now the thread two will come and it will put a lock on V1. again it will check for availability, after taking a lock because vehicle might be updated by some other thread while it is waiting.
So now, for this v1 already R1 is present for the 1st December. So, check for availability would fail. It will return false and it will not be able to create a new reservation.
So we need vehicle wise locking functionality, and that's where in the vehicle inventory manager I have added this locking behavior.

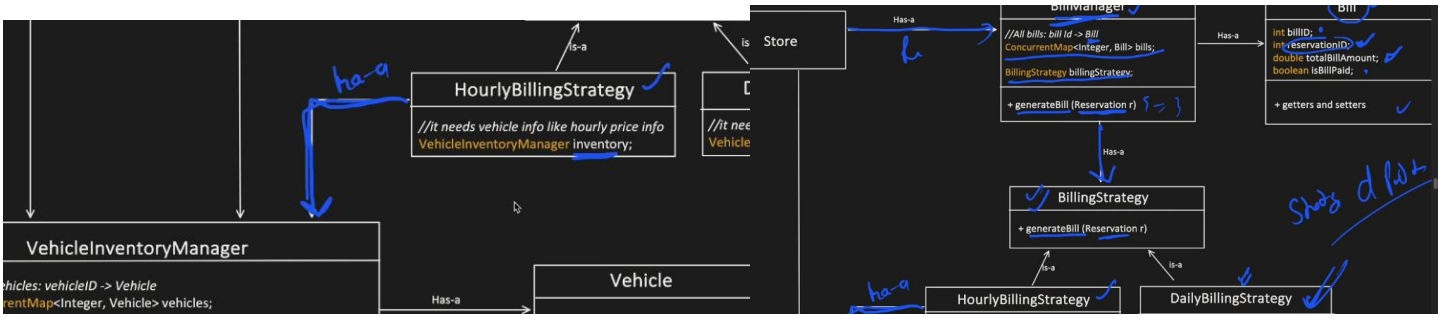


5. Bill

- Bill hold info like Reservation id like this bill is associated with which reservation. We don't need full object.
- Also, Bill is just a POJO class which is not smart enough. So we need:
  BillManager, which handles all the bills and their lifecycle.

  Store, use this BillManager to *generateBill* for the given reservation.

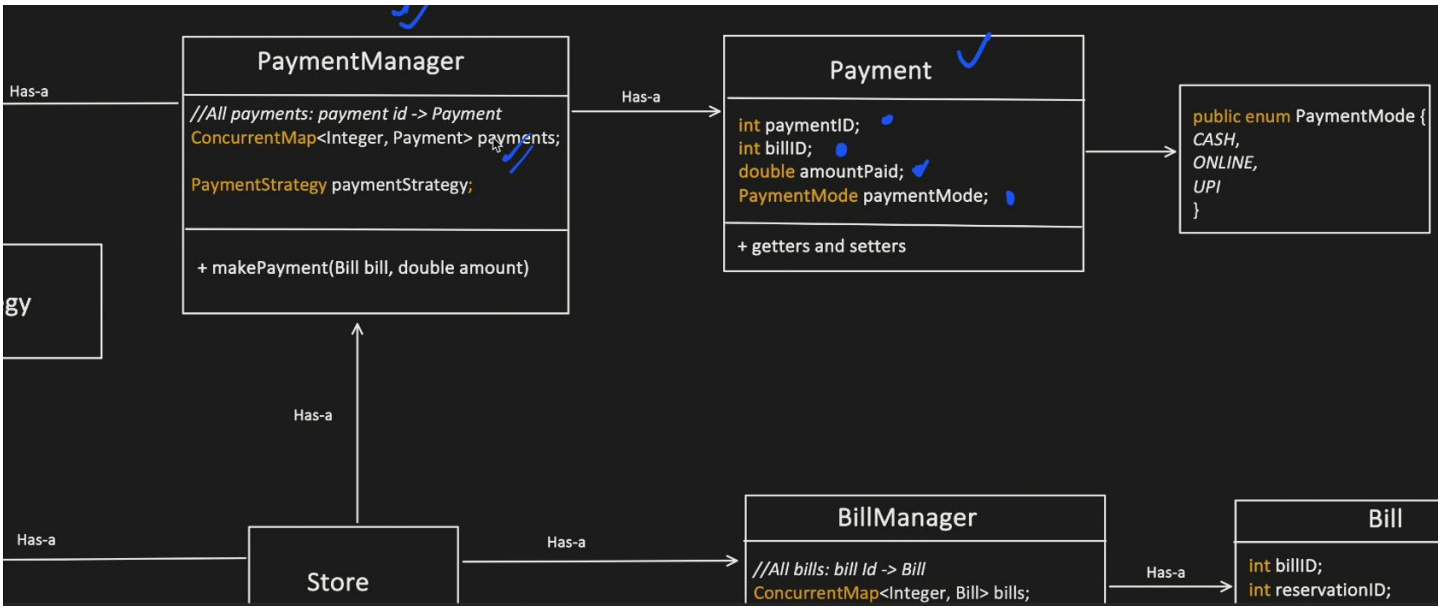


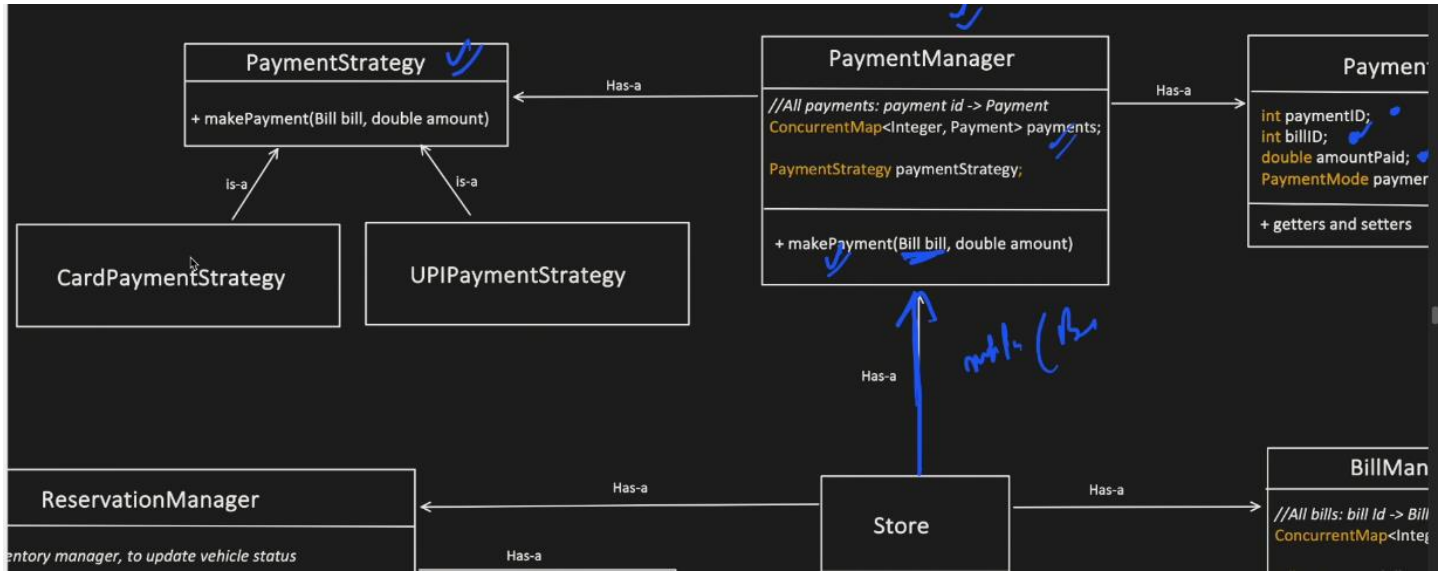


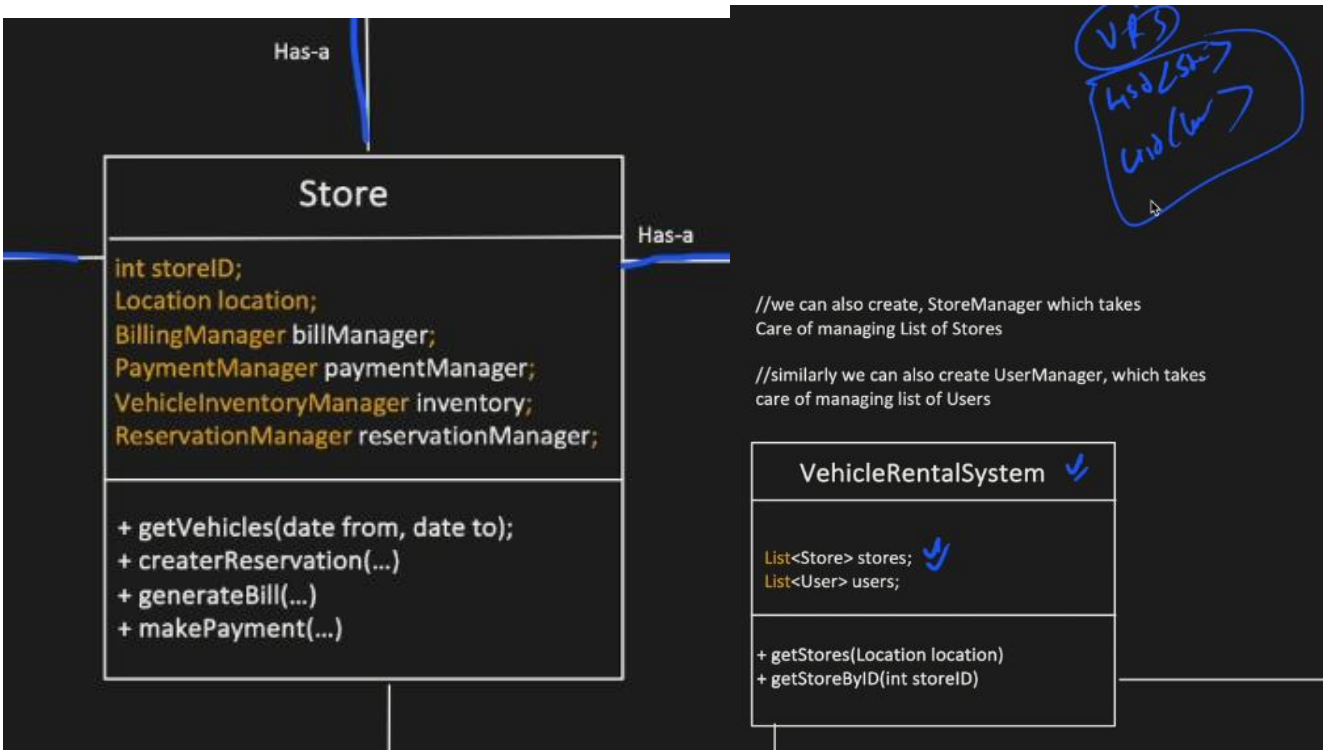
6. Payment

- Payment hold info like Bill id like this payment is associated with which bill. We don't need full bill object.
- Also, Payment is just a POJO class which is not smart enough. So we need: PaymentManager, which handles all the payments and their lifecycle.

  Store, use this PaymentManager to *make payment* for the given bill.







Orchestrator – Store

```
public class Demo {

    public static void main(String[] args) throws Exception {

        System.out.println("\n===== LLD: Car Rental System Demo =====");

        VehicleRentalSystem rentalSystem = new VehicleRentalSystem();

        // -------------------------------------------------------
        // 1. Create Stores in System
        // -------------------------------------------------------

        Location store1Location = new Location(
                45,
                "Area1",
                "City1",
                "State1",
                "India",
                12345
        );
        Store store1 = new Store(1001, store1Location);
        rentalSystem.addStore(store1);
```

```
        rentalSystem.addStore(store1);

        // -------------------------------------------------------
        // 2. Create Users
        // -------------------------------------------------------

        User user1 = new User(801, "SJ", "DL2022GDG556690");
        User user2 = new User(802, "DJ", "DL2017DHW9090765231");

        rentalSystem.addUser(user1);
        rentalSystem.addUser(user2);

        // -------------------------------------------------------
```

```
        // -------------------------------------------------------
        // 3. Add vehicles to store inventory
        // -------------------------------------------------------

        Vehicle v1 = new Vehicle(1, "DL1234", VehicleType.FOUR_WHEELER);
        v1.setDailyRentalCost(1100);

        Vehicle v2 = new Vehicle(2, "DL5678", VehicleType.FOUR_WHEELER);
        v2.setDailyRentalCost(1400);

        store1.getInventory().addVehicle(v1);
        store1.getInventory().addVehicle(v2);
```

```
        // -------------------------------------------------------
        // 4. User selects store and searches vehicles
        // -------------------------------------------------------

        Store selectedStore = rentalSystem.getStore(1001);

        LocalDate fromDate = LocalDate.of(2025, 12, 5);
        LocalDate toDate   = LocalDate.of(2025, 12, 7);

        System.out.println("\nAvailable vehicles from " + fromDate + " to " + toDate + ":");

        for (Vehicle v : selectedStore.getVehicles(
                VehicleType.FOUR_WHEELER,
                fromDate,
                toDate
        )) {
            System.out.println(" - " + v.getVehicleID() + ": " + v.getVehicleType());
        }
```

```java
// ---------------------------------------------------------
// 5. User creates reservation
// ---------------------------------------------------------
System.out.println("\nCreating reservation...");

Reservation reservation =
        selectedStore.createReservation(
                1,                  // vehicle ID
                user1,
                fromDate,
                toDate,
                ReservationType.DAILY
        );

System.out.println("Reservation created with ID: " + reservation.getReservationId());
```

```java
// ---------------------------------------------------------
// 6. User starts the trip
// ---------------------------------------------------------

System.out.println("\nStarting trip...");
selectedStore.startTrip(reservation.getReservationId());

// ---------------------------------------------------------
// 7. User submits the vehicle
// ---------------------------------------------------------

System.out.println("Submitting vehicle...");
selectedStore.submitVehicle(reservation.getReservationId());
```

```java
// ---------------------------------------------------------
// 8. Generate bill
// ---------------------------------------------------------

System.out.println("\nGenerating bill...");

Bill bill = selectedStore.generateBill(
    reservation.getReservationId(),
    new DailyBillingStrategy(selectedStore.getInventory())
);

System.out.println("Bill ID: " + bill.getBillId());
System.out.println("Bill Amount: " + bill.getTotalBillAmount());
```

```java
// ---------------------------------------------------------
// 9. Make payment
// ---------------------------------------------------------

System.out.println("\nProcessing Payment...");

Payment payment = selectedStore.makePayment(
    bill,
    new UPIPaymentStrategy(),
    bill.getTotalBillAmount()
);

System.out.println("\n===== PAYMENT RECEIPT =====");
System.out.println("Payment ID:  " + payment.getPaymentId());
System.out.println("Paid Amount:  " + payment.getAmountPaid());
System.out.println("Payment Mode: " + payment.getPaymentMode());
System.out.println("Payment Date: " + payment.getPaymentDate());
System.out.println("============================");
```

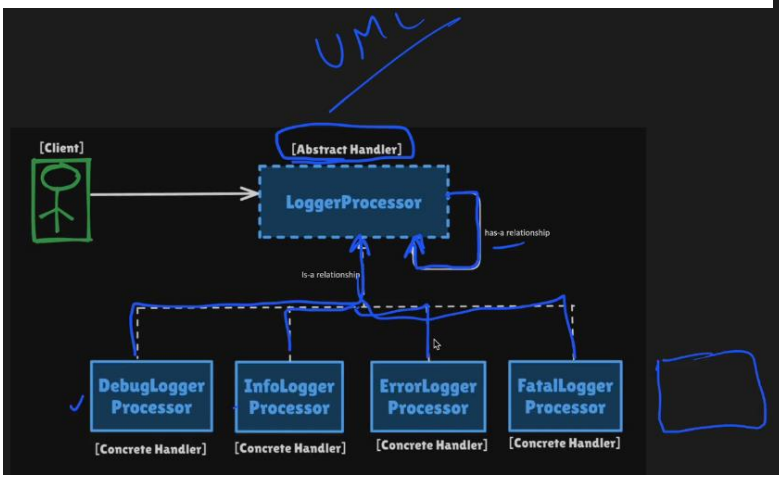
```
===== LLD: Car Rental System Demo =====

Available vehicles from 2025-12-05 to 2025-12-07:
 - 1: FOUR_WHEELER
 - 2: FOUR_WHEELER

Creating reservation...
Reservation created with ID: 20000

Starting trip...
Submitting vehicle...

Generating bill...
Bill ID: 5000
Bill Amount: 3300.0

Processing Payment...

===== PAYMENT RECEIPT =====
Payment ID: 9000
Paid Amount: 3300.0
Payment Mode: UPI
Payment Date: Wed Dec 03 10:52:14 IST 2025
============================
```

Refer : https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/interviewquestions/carrental?ref_type=heads

12. Chain Of Responsibility Design Pattern (Behavioral Design Pattern)

The Chain of Responsibility pattern is a behavioral design pattern that passes requests along a chain of handlers. Each handler decides either to process the request or pass it to the next handler in the chain.

Real-life Examples

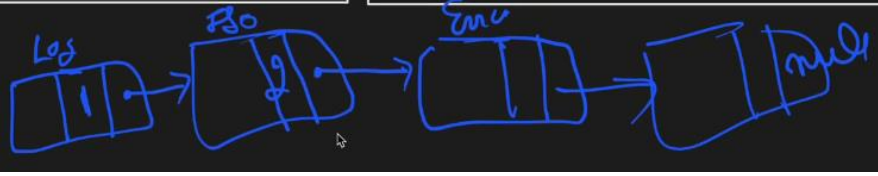



```java
// Abstract Logger class - defines the chain structure
public abstract class LogProcessor {
  public static final int DEBUG = 1;
  public static final int INFO = 2;
  public static final int ERROR = 3;
  public static final int FATAL = 4;
  int level;

  LogProcessor nextLoggerProcessor;

  public void logMessage(int level, String message) {
    if (this.level == level) {
      write(message);
    }
    // Pass to next handler in chain if exists
    if (this.nextLoggerProcessor != null) {
        this.nextLoggerProcessor.logMessage(level, message);
    }
  }

  abstract protected void write(String message);
}
```

```java
Concrete handler for DEBUG level
blic class DebugLogProcessor extends LogProcessor {

public DebugLogProcessor(int level, LogProcessor nextLogProcessor) {
  this.level = level;
  this.nextLoggerProcessor = nextLogProcessor;
}

@Override
protected void write(String message) {
  System.out.println("DEBUG: " + message);
}
```

```java
// Concrete handler for INFO level
public class InfoLogProcessor extends LogProcessor {

public InfoLogProcessor(int level, LogProcessor nextLogProcessor) {
    this.level = level;
    this.nextLoggerProcessor = nextLogProcessor;
  }

  @Override
  protected void write(String message) {
    System.out.println("INFO: " + message);
  }
}
```

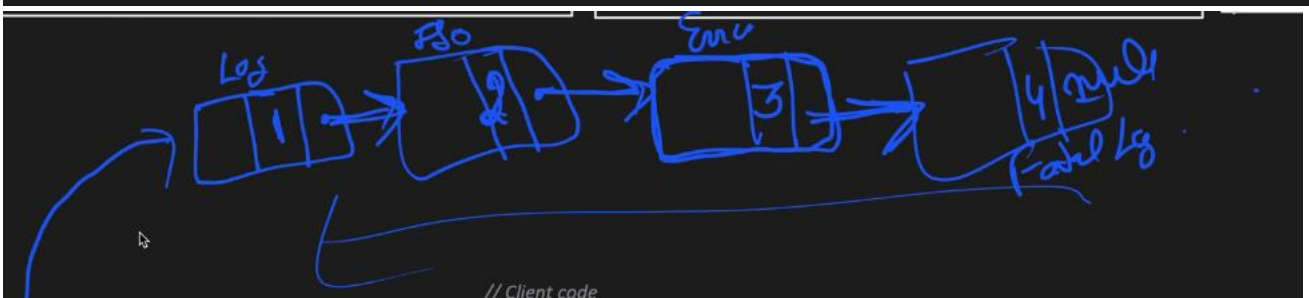
```java
// Concrete handler for ERROR level
public class ErrorLogProcessor extends L

public ErrorLogProcessor(int level, LogPro
    this.level = level;
    this.nextLoggerProcessor = nextLogP
  }

  @Override
  protected void write(String message) {
    System.out.println("ERROR: " + mess
  }
}
```



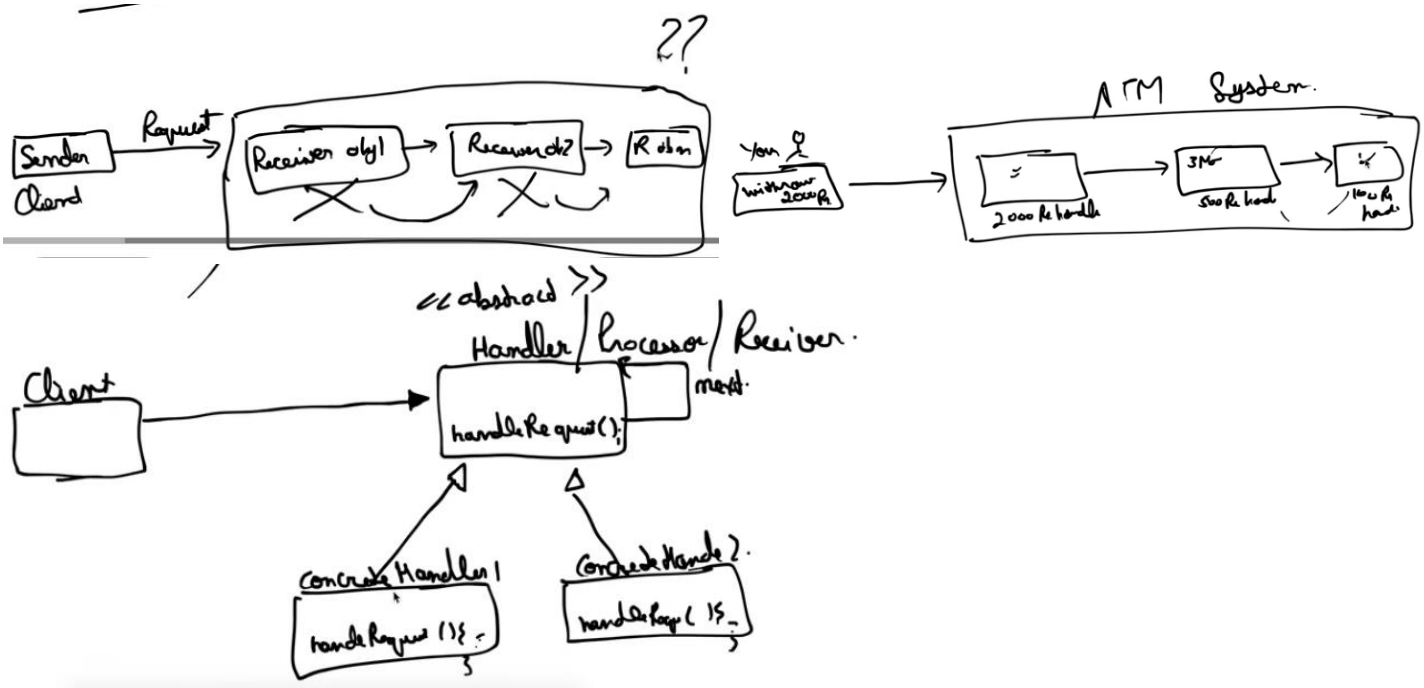
```
gProcessor {

cessor nextLogProcessor) {

rocessor;


ge);
```

```
// Concrete handler for ERROR level
public class ErrorLogProcessor extends LogProcessor {

public ErrorLogProcessor(int level, LogProcessor nextLogProcessor) {
      this.level = level;
     this.nextLoggerProcessor = nextLogProcessor;
   }

   @Override
   protected void write(String message) {
      System.out.println("ERROR: " + message);
   }
}
```

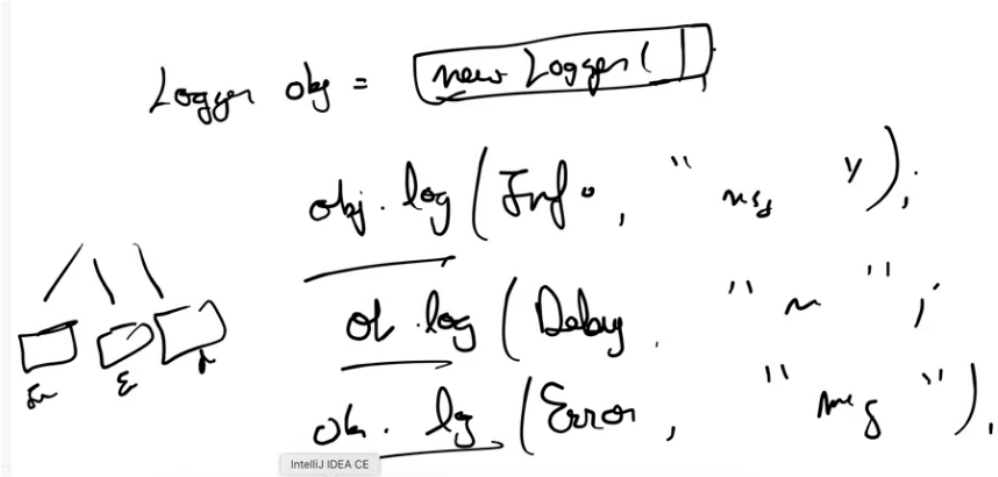
```
// Concrete handler for FATAL level
public class FatalLogProcessor extends LogProcessor {

   public FatalLogProcessor(int level, LogProcessor nextLogProcessor) {
      this.level = level;
     this.nextLoggerProcessor = nextLogProcessor;
   }

   @Override
   protected void write(String message) {
      System.out.println("FATAL: " + message);
   }
}
```




```
// Client code
public class LoggerDemo {

   public static void main(String[] args) {

      // Create Chain: DEBUG -> INFO -> ERROR -> FATAL
       LogProcessor fatalLogger = new FatalLogProcessor(LogProcessor.FATAL, null); // 4
       LogProcessor errorLogger = new ErrorLogProcessor(LogProcessor.ERROR, fatalLogger); // 3
       LogProcessor infoLogger = new InfoLogProcessor(LogProcessor.INFO, errorLogger ); // 2
       LogProcessor logger= new DebugLogProcessor(LogProcessor.DEBUG, infoLogger ); // 1


       // start from the head of the chain i.e. DEBUG
        logger.logMessage(LogProcessor.DEBUG, "This is a debug message");

        logger.logMessage(LogProcessor.INFO, "This is an info message");

         logger.logMessage(LogProcessor.ERROR, "This is an error message");

         logger.logMessage(LogProcessor.FATAL, "This is a fatal message");
   }
}
```



Example :

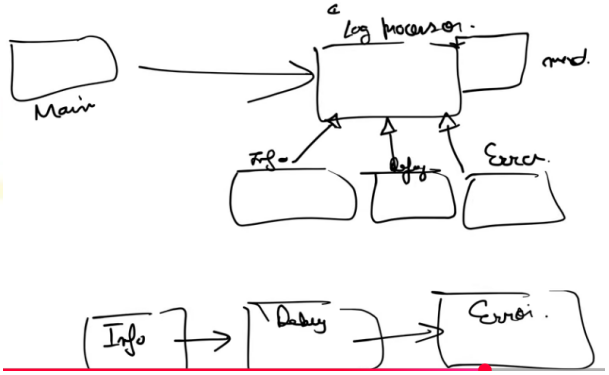



```java
public abstract class LogProcessor {

    public static int INFO = 1;
    public static int DEBUG = 2;
    public static int ERROR = 3;

    LogProcessor nextLoggerProcessor;

    LogProcessor(LogProcessor loggerProcessor) {

        this.nextLoggerProcessor = loggerProcessor;

    }

    public void log(int logLevel, String message) {

        if (nextLoggerProcessor != null) {
            nextLoggerProcessor.log(logLevel, message);
        }
    }
}
```

```java
public class InfoLogProcessor extends LogProcessor{

    InfoLogProcessor(LogProcessor nexLogProcessor) { super(nexLogProcessor); }

    public void log(int logLevel,String message){

        if(logLevel == INFO) {
            System.out.println("INFO: " + message);
        } else{

            super.log(logLevel, message);
        }

    }
}
```

```java
public class ErrorLogProcessor extends LogProcessor{

    ErrorLogProcessor(LogProcessor nexLogProcessor) { super(nexLogProcessor); }

    public void log(int logLevel,String message){

        if(logLevel == ERROR) {
            System.out.println("ERROR: " + message);
        } else{

            super.log(logLevel, message);
        }

    }
}
```

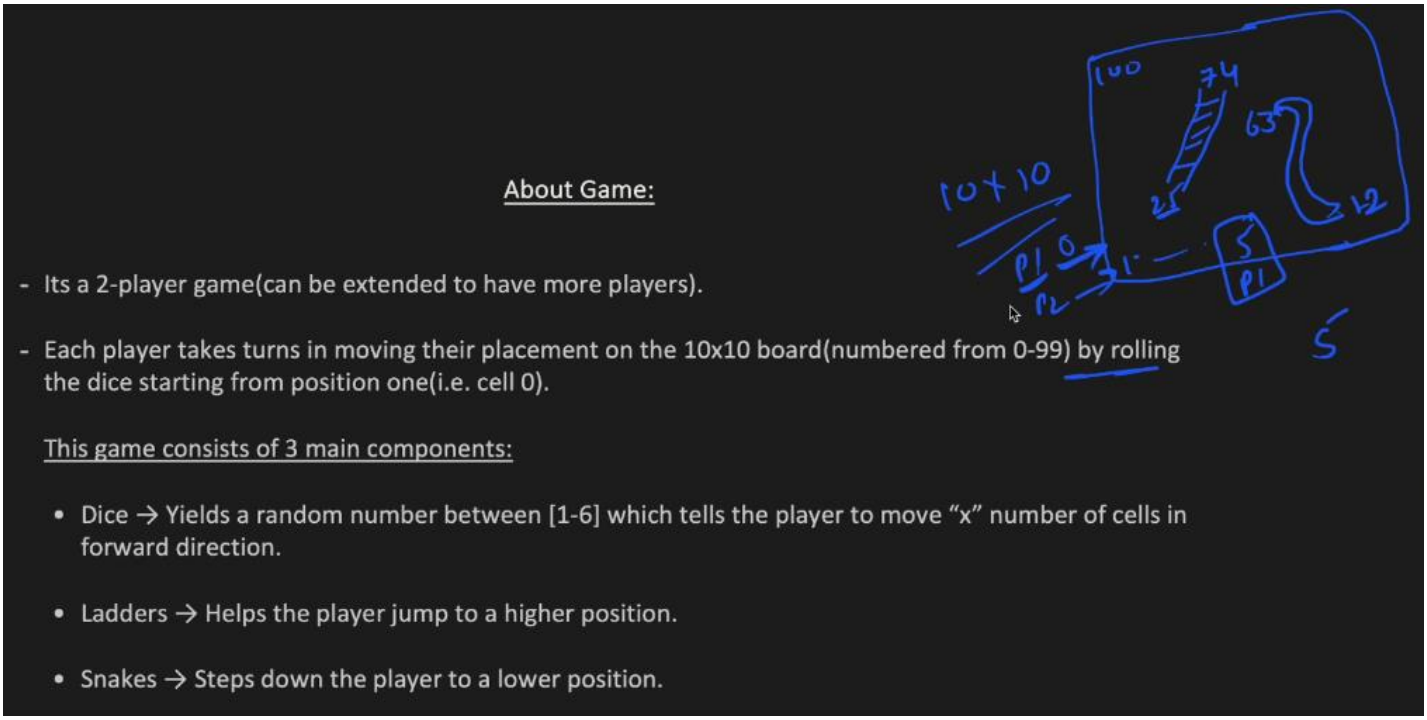
```java
public class DebugLogProcessor extends LogProcessor{

    DebugLogProcessor(LogProcessor nexLogProcessor) { super(nexLogProcessor); }

    public void log(int logLevel,String message){

        if(logLevel == DEBUG) {
            System.out.println("DEBUG: " + message);
        } else{

            super.log(logLevel, message);
        }

    }
}
```

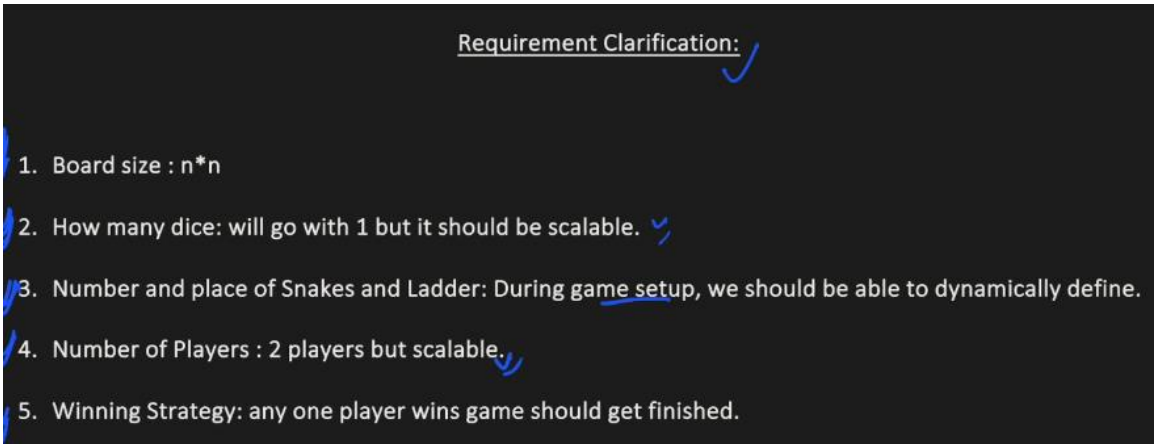



```java
public class Main {

    public static void main(String args[]) {

        LogProcessor logObject = new InfoLogProcessor(new DebugLogProcessor(new ErrorLogProcessor( nexLogProcessor: null)));

        logObject.log(LogProcessor.ERROR,  message: "exception happens");
        logObject.log(LogProcessor.DEBUG,  message: "need to debug this ");
        logObject.log(LogProcessor.INFO,  message: "just for info ");

    }
}
```

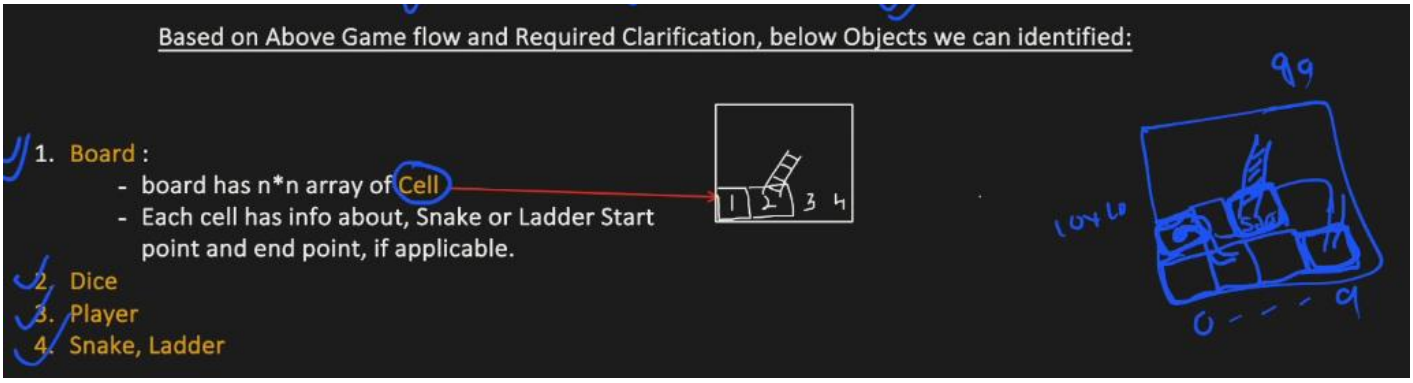
13. LLD of Snake n Ladder Game

https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/interviewquestions/snakeNladder?ref_type=heads



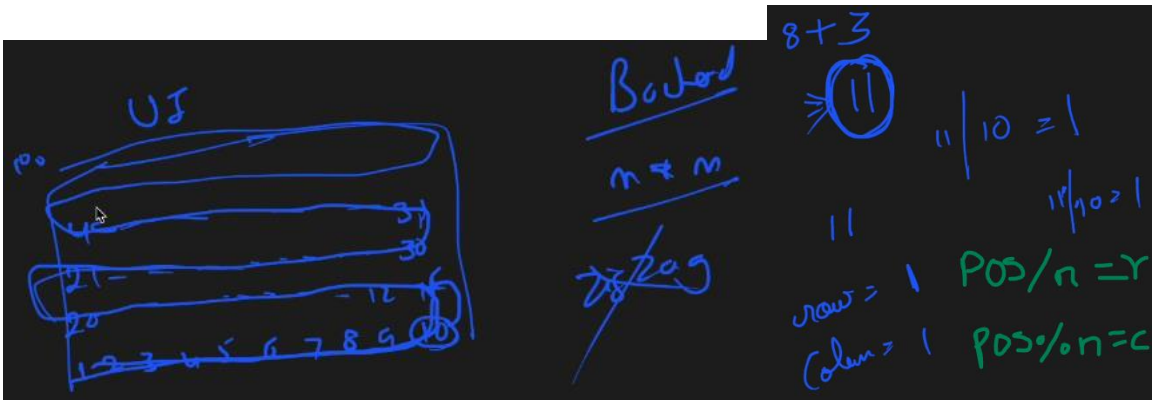


Every cell needs to store information of snake or ladder in it and the next point to go



The board is not a zigzag form; it's just a way of representation at the UI.
So, at the back end when we create a matrix let's say n by n, we don't have to handle this zigzag.

At UI it looks like players are moving in a zigzag form. But the players are moving in a sequential based on the number



Lets start the UML:

1. Player:

- Holds variables to save the name or id.
- Player current position on the board.

the range of current position is 0-99 for the player

Player
+ id: String + currentPosition: int

2. Dice:

- Holds DiceCount, which tells with how many dice we are playing the game.
- Player current position on the board.
- rollDice() method simply, find the number between 1 and 6 for N (DiceCount) times and sum it up.

Dice
+ diceCount: int
+ void rollDice()

Player
+ id: String + currentPosition: int

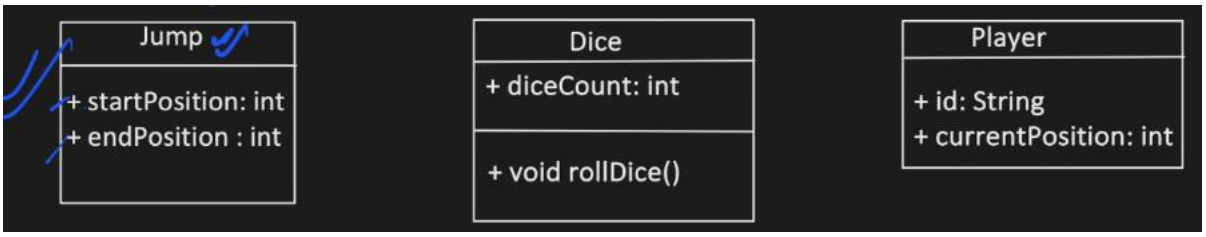
3. Snake and Ladder:

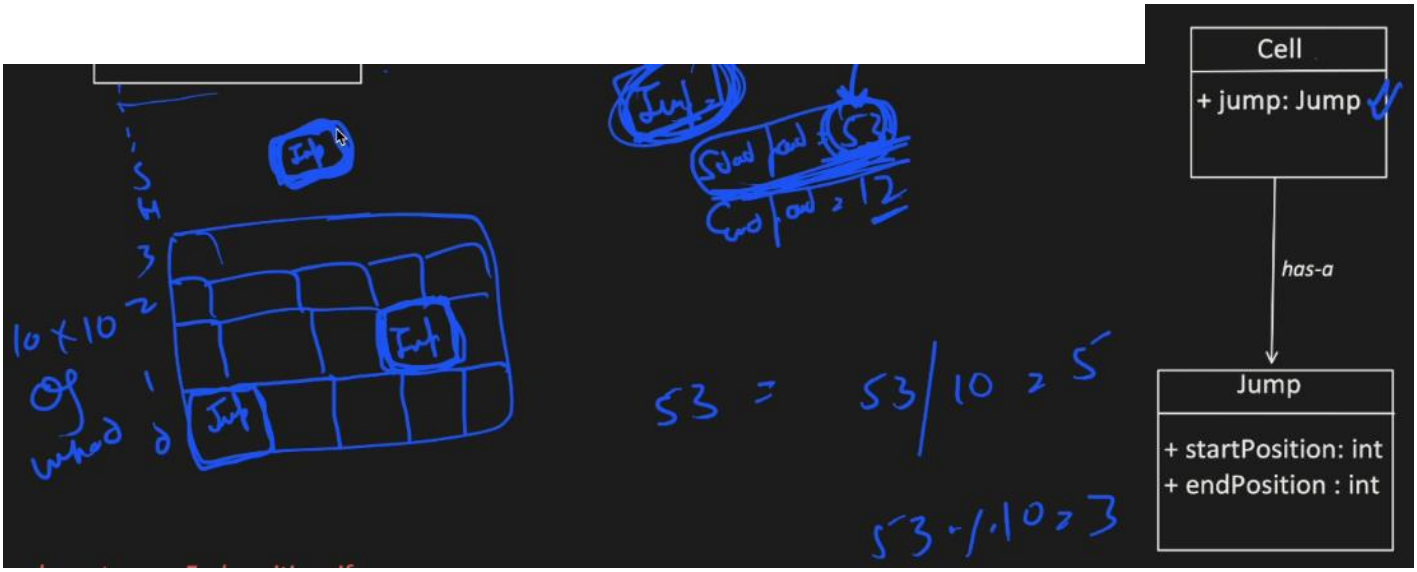
- Either we can have 2 classes one for Snake and one for Ladder.

Snake	Ladder
int startPosition;	int startPosition;
int endPosition;	int endPosition;

- But both the class looks same. Only difference is:
  - Snake: Start position is greater than end position.
  - Ladder: End position is greater than start position.

So, we can create only 1 class i.e Jump.
As, both Snake and Ladder work is to provide the Jump, whether Jump to high position or Jump to Low.

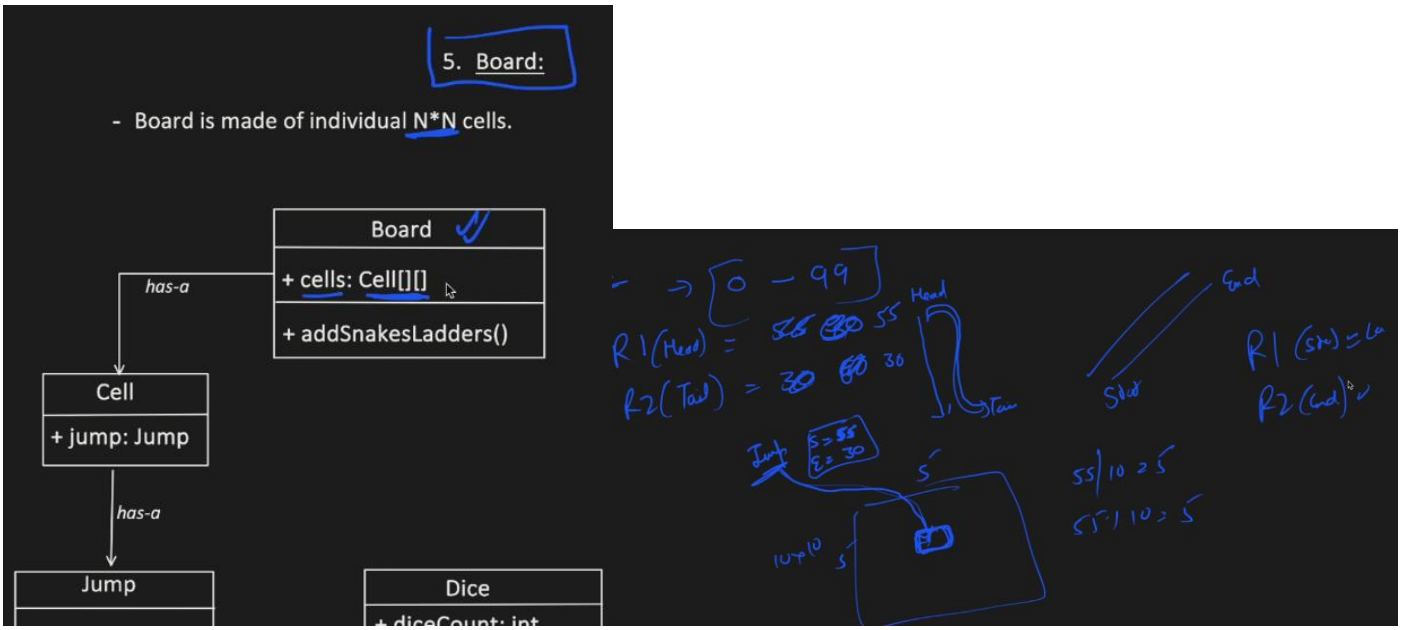




4. Cell:

- Board is made of individual N*N cells.

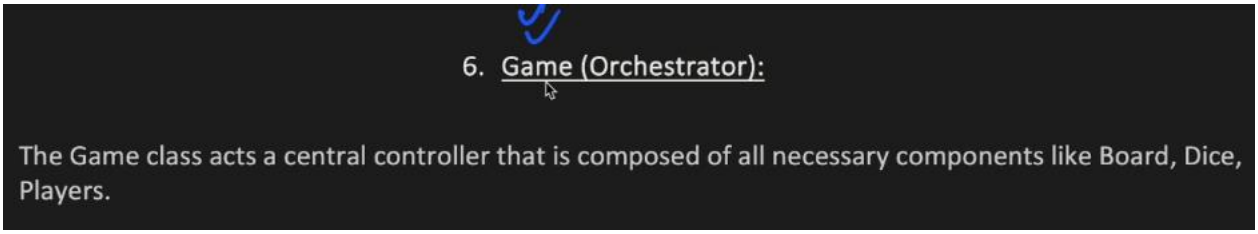
- Each cell has info about:
  - Jump (represent either Snake or Ladder, we don't care)
    When player reach a particular cell, and if Jump is not Null, we will move the player to new End position. if its Ladder, player will go to higher position and if its Snake, it will go to lower position.

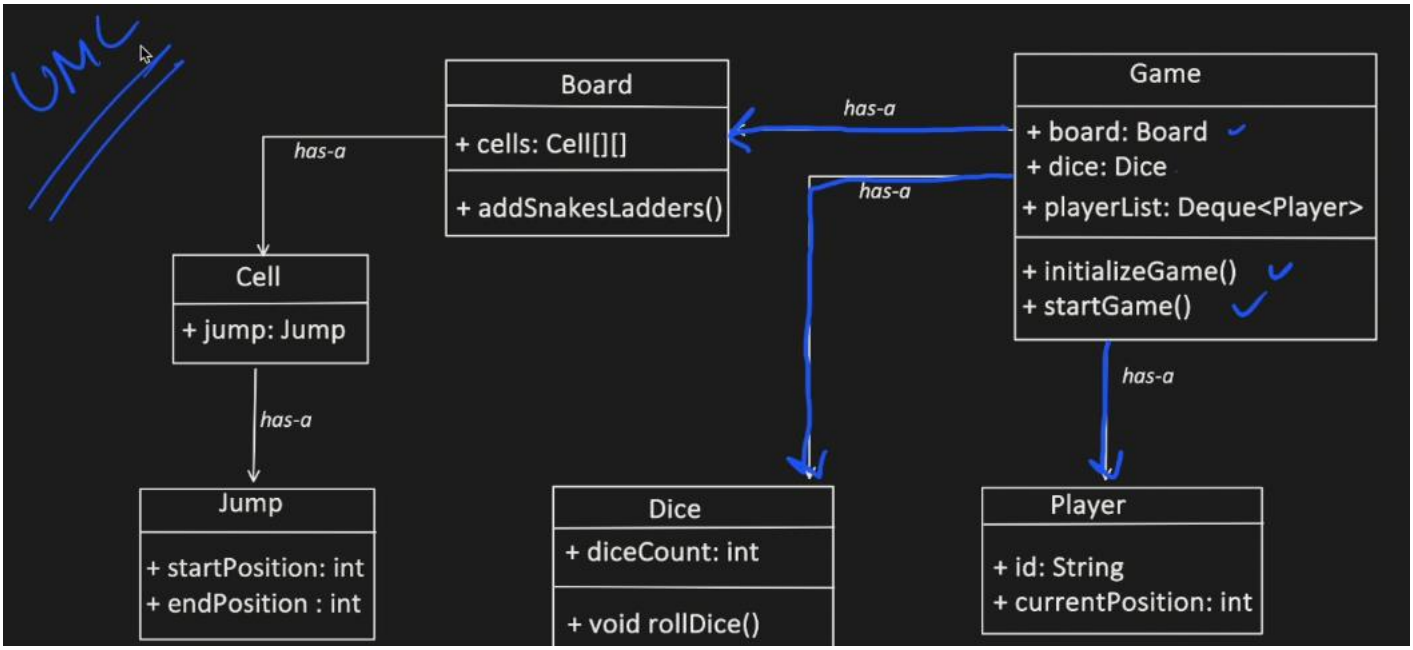


Some logic needs to be taken care while creating the snakes and ladder:

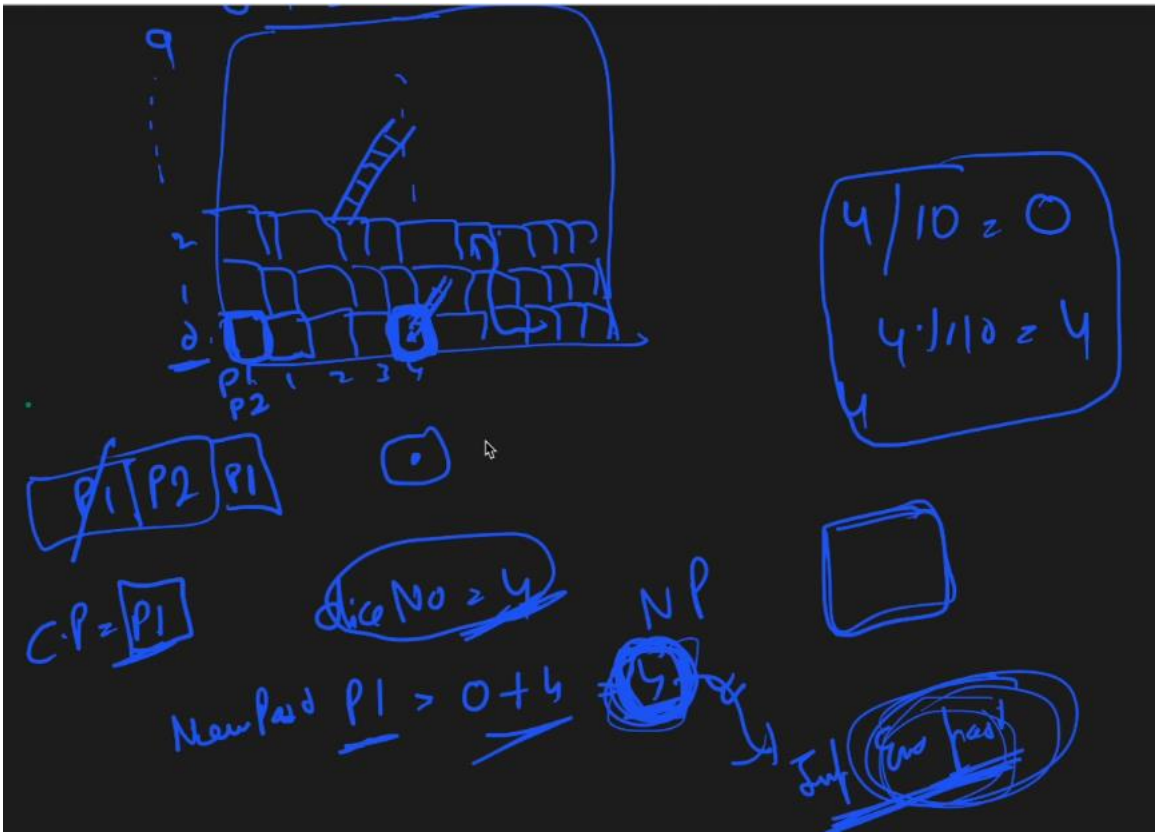
At each cell, there can be only one jump object. A jump object can represent either a snake or a ladder or a null. The jump object is assigned to cells at start. A cell can be empty, or a snakehead or snake tail or ladder start or ladder end. Since a cell has only one jump object, Snake head - ladder start cannot co-exist. There should be some logic to avoid snakehead - ladder end pair and snake tail - ladder start pair.

Snake head should not be present at 99, the last position. If present, this game will never end.





DQ of players - alternate chance to players




```
SnakeNLadderDemo
/Users/shrayanshjain/Library/Java/JavaVirtualMach
Player turn:Player-1 current position is: 0
Player turn:Player-1 new Position is: 2
Player turn:Player-2 current position is: 0
Player turn:Player-2 new Position is: 6
Player turn:Player-1 current position is: 2
Player turn:Player-1 new Position is: 4
Player turn:Player-2 current position is: 6
Player turn:Player-2 new Position is: 10
Player turn:Player-1 current position is: 4
Player turn:Player-1 new Position is: 10
Player turn:Player-2 current position is: 10
Player turn:Player-2 new Position is: 13
```

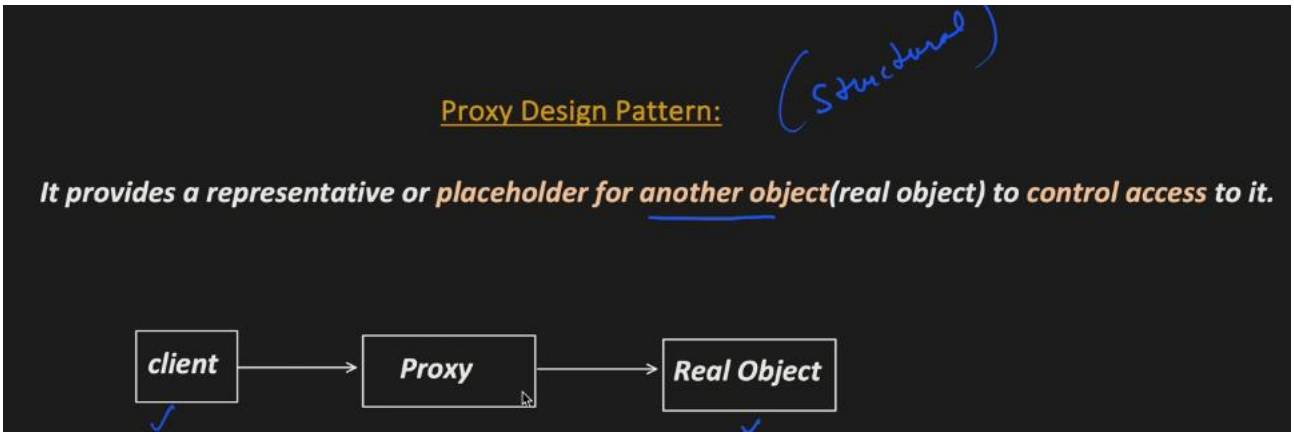
```
Player turn:Player-2 new Position is: 59
Player turn:Player-1 current position is: 75
Player turn:Player-1 new Position is: 77
Player turn:Player-2 current position is: 59
Player turn:Player-2 new Position is: 61
Player turn:Player-1 current position is: 77
[+] Jump done by: Snake
Player turn:Player-1 new Position is: 35
Player turn:Player-2 current position is: 61
Player turn:Player-2 new Position is: 63
Player turn:Player-1 current position is: 35
Player turn:Player-1 new Position is: 38
Player turn:Player-2 current position is: 63
```

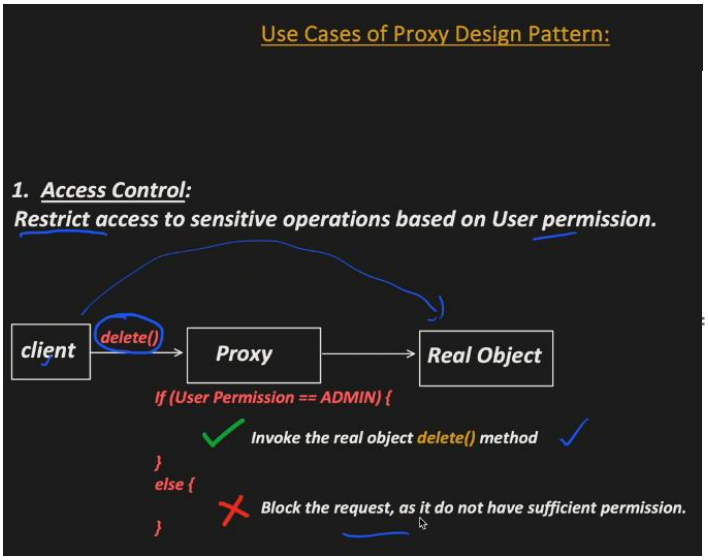
```
Player turn:Player-2 new Position is: 46
Player turn:Player-1 current position is: 53
Player turn:Player-1 new Position is: 56
Player turn:Player-2 current position is: 46
Player turn:Player-2 new Position is: 48
Player turn:Player-1 current position is: 56
[+] Jump done by: Ladder
Player turn:Player-1 new Position is: 77
Player turn:Player-2 current position is: 48
Player turn:Player-2 new Position is: 49
Player turn:Player-1 current position is: 77
Player turn:Player-1 new Position is: 80
```

```
Player turn:Player-1 current position is: 52
Player turn:Player-1 new Position is: 57
Player turn:Player-2 current position is: 95
Player turn:Player-2 new Position is: 96
Player turn:Player-1 current position is: 57
Player turn:Player-1 new Position is: 63
Player turn:Player-2 current position is: 96
Player turn:Player-2 new Position is: 100

===> The Winner is:Player-2

Process finished with exit code 0
```

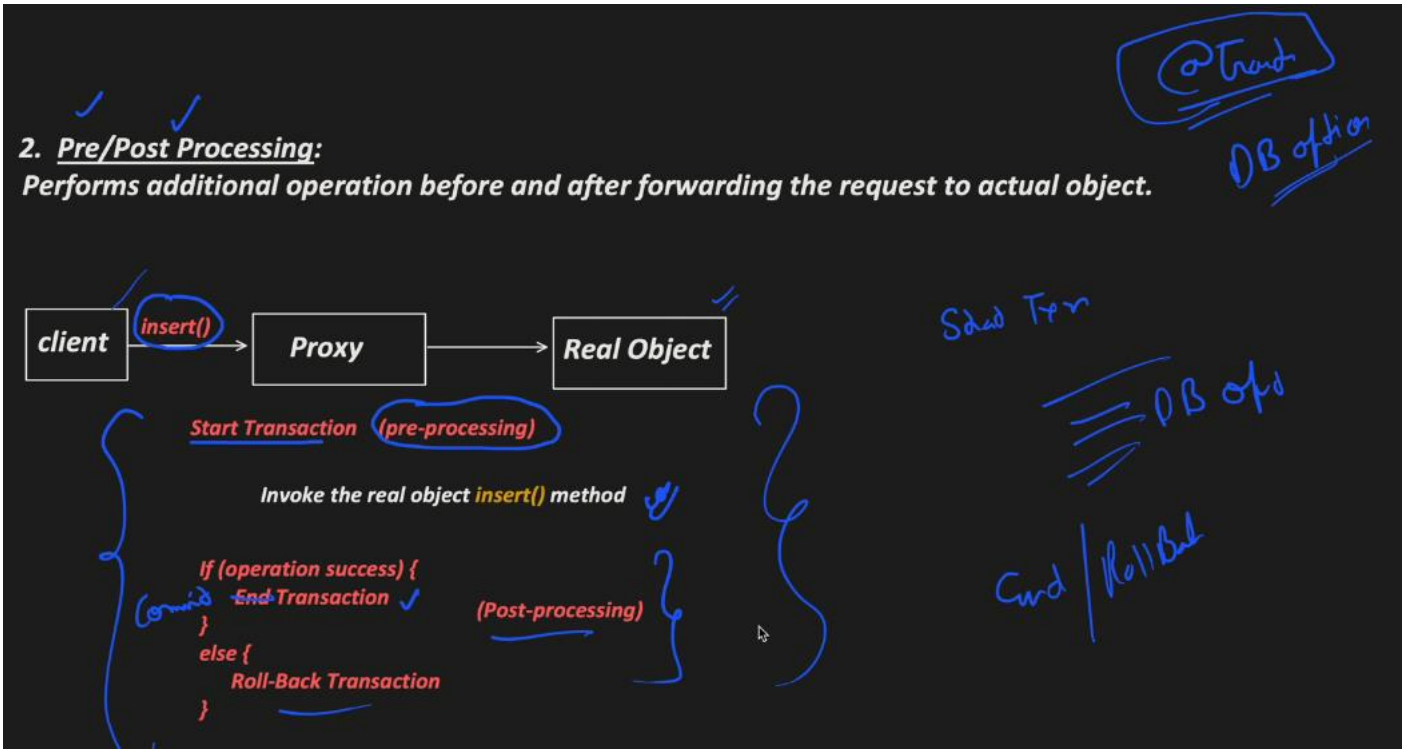
Refer : https://gitlab.com/shrayansh8/lld-lowleveldesign/-/blob/main/src/main/java/com/conceptcoding/interviewquestions/snakeNladder/Game.java?ref_type=heads

14. Proxy Design Pattern (Structural Pattern)

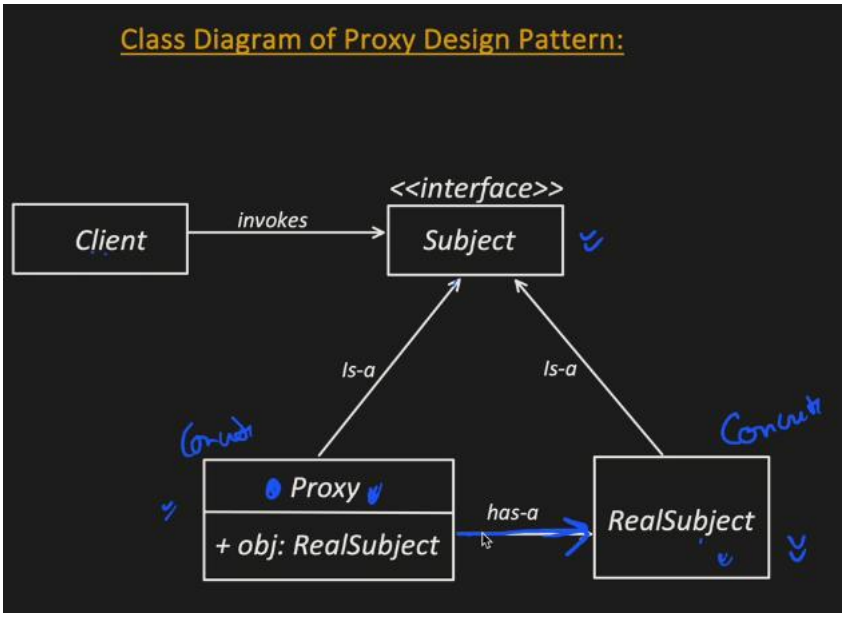


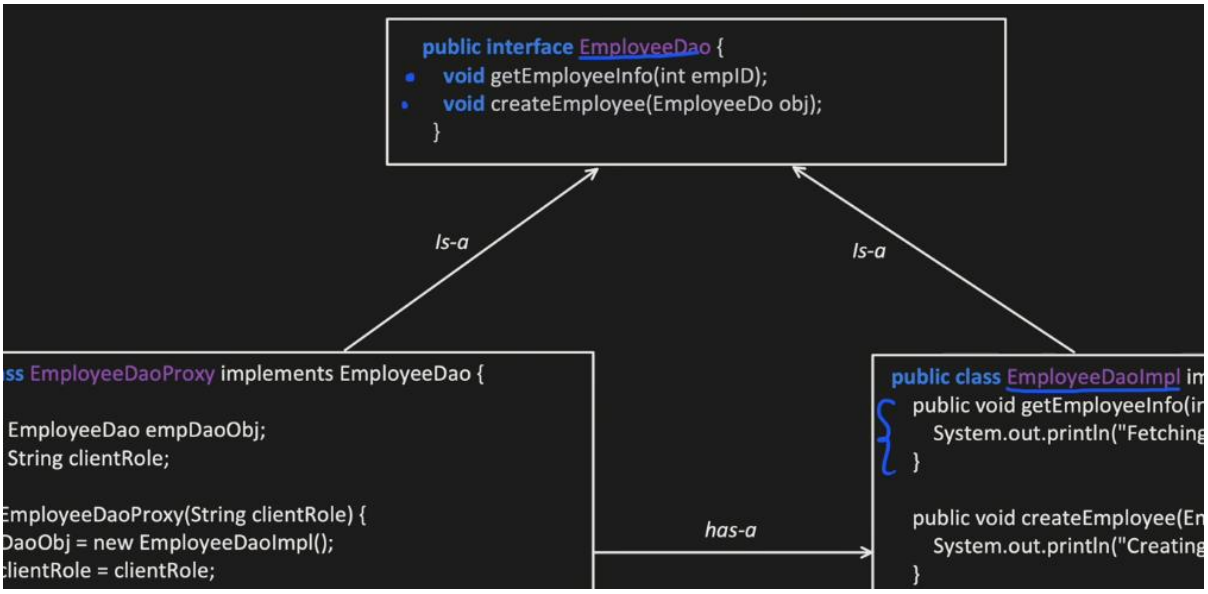


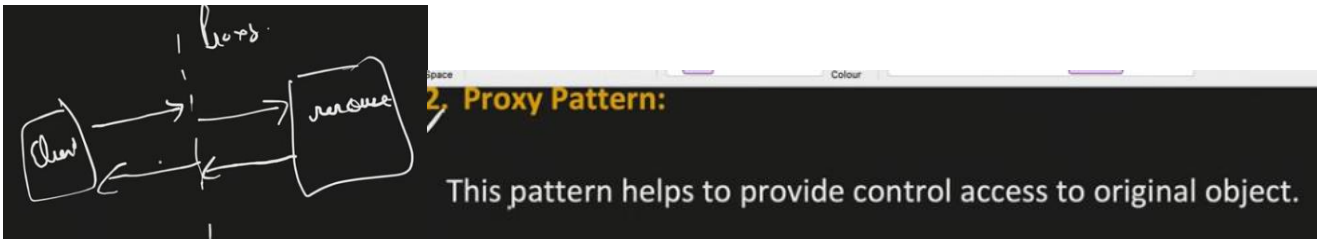
When we are trying to update the DB internally, we have to create a transaction. We must start the transaction and then perform a DB operation, and then we must end or roll back based on the success of the transaction.
Generally, in spring boot, when we use at the rate transaction annotation, we just perform db operation. the DB operation which needs to be performed is just annotated with at the rate(@) annotation.
Internally it makes use of proxy design pattern.
So it does some pre-processing and post-processing.
So instead of directly calling the DB operations there is a proxy class got invoked first which starts the transaction which is nothing but pre-processing. then whatever the DB operation you wanted to perform, it does that. After the post-processing, if the operation is successful then commit
the transaction, else roll back the transaction.
So, this is a very good example for pre and processing use case of a proxy design pattern.

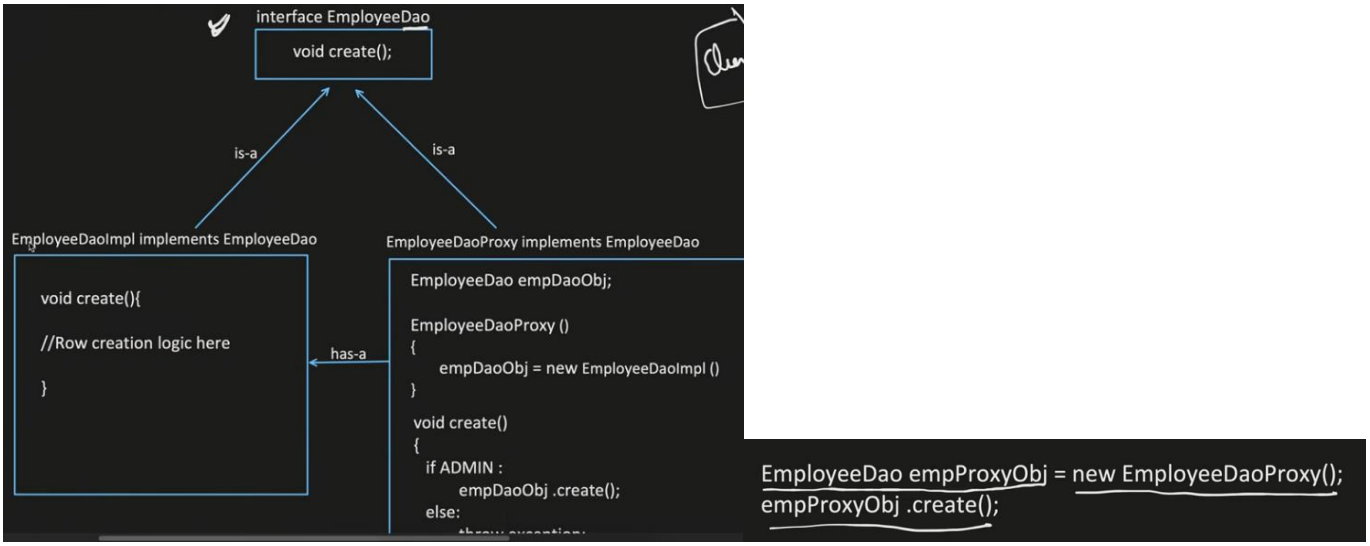


concrete class - real object. The client is interested in accessing the real object.
In the proxy design pattern, we also create one more child class of subject which is a Proxy subject- concrete class. This proxy has a reference for this real subject. Because now the client will never talk directly with the real subject. It talks with the proxy, and the proxy needs a reference so that it can further invoke the real subject.


Class Diagram of Proxy Design Pattern:
<<interface>>
Client
invokes
Subject
Is-a
Is-a
Proxy
+ obj: RealSubject
has-a
RealSubject


public interface EmployeeDao {
void getEmployeeInfo(int empID);
void createEmployee(EmployeeDo obj);
}
Is-a
Is-a
ss EmployeeDaoProxy implements EmployeeDao {
EmployeeDao empDaoObj;
String clientRole;
EmployeeDaoProxy(String clientRole) {
DaoObj = new EmployeeDaoImpl();
lientRole = clientRole;
has-a
public class EmployeeDaoImpl im
public void getEmployeeInfo(ir
System.out.println("Fetching
}
public void createEmployee(En
System.out.println("Creating
}


2. Proxy Pattern:
This pattern helps to provide control access to original object.


interface EmployeeDao
void create();
is-a
is-a
EmployeeDaoImpl implements EmployeeDao
void create(){
//Row creation logic here
}
has-a
EmployeeDaoProxy implements EmployeeDao
EmployeeDao empDaoObj;
EmployeeDaoProxy ()
{
empDaoObj = new EmployeeDaoImpl ()
}
void create()
{
if ADMIN :
empDaoObj .create();
else:
EmployeeDao empProxyObj = new EmployeeDaoProxy();
empProxyObj .create();